

# **Jahangirnagar University**



## **Assignment Topic**

---

Balancing Act: Strategic Effort Estimation for a University-Wide Student Information System (Project Polaris)

Course Title: Software Project Management and Quality Assurance  
Course Code: PMSCS 660

### **SUBMITTED TO**

**Dr. Mohammad Zahidur Rahman**  
Professor, Department of Computer Science and Engineering

### **SUBMITTED BY**

**Md. Habibur Rahman**  
Id No: CSE202601013  
Department of Computer Science and Engineering

Date: 09<sup>th</sup> July 2026

# Balancing Act: Strategic Effort Estimation for a University (JU) wide Student Information System (Project Polaris):

## 1. Introduction:

Software effort estimation is a critical part of project management, helping determine the time, budget, and resources needed. Accurate estimation improves planning and risk management, while poor estimation inevitably leads to budget overruns and missed deadlines.

Project Polaris is a massive step for Janghirmagar University, aiming to replace the monolithic legacy student information system (SIS) with a modern web platform. We must balance the "iron platform triangle" of project management: delivering all features within 15 months deadline, staying on budget



and ensuring top-notch security. This assignment applies different estimation techniques to replace guesswork with a solid, data-driven planning strategy.

## 2. Part 1: Critical Analysis of Estimation Methods:

### 2.1: Bottom-up Estimation:

This method breaks the project into tiny manageable tasks using a work break down structure (WBS) and adds them up for a total estimates. For polaris, it is useful given the three distinct modules. However building is a comprehensive WBS for a project this massive is incredibly time consuming and complicated by changing requirements. Therefore, bottom-up estimation is best reserved exclusively for high risk components - like the legacy data migration



security protocols and testing.

## 2.2: Top-Down (parametric) Estimation:

This method relies on mathematical models like COCOMO to calculate effort based on lines of code (KLOC) and productivity factors, while applicable to the Administrative Backend, COCOMO struggles to measure the cognitive work required for business logic, workflow rules and strict security features. It provides a good initial features — g but must be combined with other methods for true accuracy.

## 2.3: Analogy-Based Estimating:

This predicts effort by comparing the current work to similar past projects (using tools) like Angel. The problem is Jahangirnagar University lacks a precedent for a project of this scale, security requirement



(Privacy Act of Bangladesh) and 20-years legacy data migration. Relying purely on analogy (like comparing it to an old library system) would yield inaccurate estimates. It should only be used for smaller generic modules.

#### 2.4 Parkinson's Law & Price-to-win:

"Price to win" forces estimates to fit a desired client budget or deadline, which is a strategic failure that leads to rushed work and scope creep.

Conversely, "Parkinson's Law" suggests work expands to fill the available time. To prevent the team from losing their sense of urgency over the 15-month timeline, we need realistic estimates and aggressive intermediate milestones not fake numbers.

### 3. Part 2: Applying parametric and functional size models:

#### 3.1 COCOMO Application for the Administrative Backend:

The Administrative Backend is estimated at 85 ~~lacs~~ KSLOC. As a mix of new code and old system integration we treat it as a "semi detached" system ( $C = 3.0$  and  $K = 1.12$ ).

Step 1: Calculating the Normal Effort

$$\text{Effort} = C \times \text{Size}^K$$

$$\text{Effort} = 3.0 \times (85)^{1.12}$$

Nominal Effort  $\approx 418.66$  Staff-months

Step 2: Calculating the Adjusted Effort

We adjust this base number using three cost drivers: low language experience



very high complexity and high reliability

$$\text{Adjusted Effort} = 418.66 \times (2.14 \times 1.34 \times 1.15)$$

$$\text{Adjusted Effort} = 418.66 \times 1.756$$

$$\text{Adjusted Effort} \approx 735.34 \text{ staff-months}$$

### step 3: Interpretation

This adjustment adds over 300 staff-months to the original estimate! This clearly shows that the real huddles for the backend aren't just code volume but the team's lack of experience combined with extreme logical complexity. COCOMO alone is too rigid to capture human elements like coordinating with faculty or engineering strict degree audit workflow.

### 3.2 Function point analysis for the Student Portal

Function point Analysis (FPA) measures system size based on logical functional requirements. We'll use the Mark II FPA method, focusing on inputs ( $N_i$ ), entry typed accessed ( $N_e$ ) and outputs ( $N_o$ )

Step 1: Mark II FP calculation (register for a course)

inputs ( $N_i$ ) = 8, Entities ( $N_e$ ) = 7  
outputs ( $N_o$ ) = 4

$$FP = (8 \times 0.58) + (7 \times 1.66) + (4 \times 0.26)$$

$$FP = 4.64 + 11.62 + 1.04$$

$$\text{Total size} = 17.3 \text{ Mark II FPs}$$



## Step 2: MARK II vs. COSMIC

MARK II is a great for standard user-facing interfaces like the student portal, However, the data integration & Reporting layer requires the COSMIC FP approach. COSMIC measures data movements (Entires, Exits, Reads and writer) making it much more accurate and massive backend engines handling heavy database queries and persistent storage operations.

## 4. part 3: Synthesis and Recommendation

### 4.1 Recommendation Hybrid Estimation Strategy

Relying on just one estimation model for a multi-faceted project

invites systematic risk. I propose a hybrid approach:

Top-Down (parametric): Use this to establish a macro-level budget and timeline for university board approval.

Bottom-up: Strictly use this for the legacy data migration and SSO authentication components. Every database mapping and sanitization script must be estimated individually to ensure data integrity.

Analogy-Based: Use past data from the university's HR portal upgrade solely to estimate standard UI/UX work in the student portal.



## 4.2 Addressing the project RISKS

The 15-month Deadline: Bottom-up estimates for critical tasks enable eight two week sprints, preventing Parkinson's Law.

Legacy data migration: Isolating this task with a bottom-up approach allocates explicit, dedicated time just for data cleansing.

No Historical Data: Relying mostly on parametric and functional models saves us from making bad comparisons to irrelevant past projects.

Privacy compliance: Factoring in the "high reliability" COCOMO multiplier ensures our schedule has built-in time

for the security testing required by the Privacy Act of Bangladesh.

### 4.3 critiquing a Hypothetical 150 Staff-Month Estimate

If the dev team handed me a draft estimate of 150 staff months, I'd heavily scrutinize it using the guidelines:

1. What are the task size drivers? Are those 150 months based on actual calculations (like our 17.3 FPs) or arbitrary guesses? If it's not backed by quantified drivers, it's invalid.

2. What productivity rates been proven? Did the estimator ignore our junior developers' low experience multiplier?



Assuming universal "high" productivity will inevitably cause delays.

3. Is there a similar previous project comparing this to an old, smaller library system catastrophically underestimates the integration complexity of polaris.

4. Have there productivity rates been proven?

can they show me empirical proof of a mixed-experienced team delivering a secure, PII-compliant system in this timeframe? Without industry benchmarks 150 staff-months is dangerously optimistic

References:

- PMSCS CSE660 class Assignment, Software Effort Estimation.
- Hughes, B. & Cottrell M. Software project Management (chapter 5)